

Tree-Based Machine Learning: Theory and Methods Companion

Detailed Reference — USDA · NRCS · National Soil Survey Center

Digital Soil Mapping Training Series

2026-02-24

Contents

1	Classification and Regression Trees (CART)	2
1.1	Conceptual Overview	2
1.2	Split Criteria	3
1.2.1	Regression Trees — Residual Sum of Squares	3
1.2.2	Classification Trees — Gini Impurity	3
1.2.3	Multi-class vs Binary Classification	4
1.3	Cost-Complexity Pruning	4
1.3.1	The Overfitting Problem	4
1.3.2	The Cost-Complexity Criterion	4
1.3.3	Reading the CP Table	4
1.3.4	The 1-SE Rule	5
1.3.5	Visualising Pruning: <code>plotcp()</code>	5
1.4	Response Transformation: <code>log1p</code> for Skewed Data	5
1.4.1	When to Transform	5
1.4.2	The <code>log1p</code> Transform	5
1.4.3	Comparing Models on the Original Scale	5
2	Random Forest	6
2.1	Bagging: The Foundation	6
2.2	Random Feature Subsampling: The RF Innovation	6
2.3	Key Parameters	6
2.3.1	<code>ntree</code> — Number of Trees	6
2.3.2	<code>mtry</code> — Features Tried per Split	7
2.3.3	<code>nodesize</code> / <code>min.node.size</code> — Minimum Leaf Size	7
2.4	Out-of-Bag (OOB) Error	7
2.5	Variable Importance	8
2.5.1	<code>%IncMSE</code> (Permutation Importance)	8
2.5.2	<code>IncNodePurity</code> (Gini Importance)	8
2.6	The Histosol Outlier Effect	8
2.7	Hyperparameter Tuning	9
2.7.1	Grid Search	9
2.7.2	<code>caret</code> + <code>ranger</code> Tuning	9
2.8	<code>randomForest</code> vs <code>ranger</code>	9
3	Feature Selection with Boruta	10
3.1	The Shadow Feature Approach	10
4	Cross-Validation	11
4.1	Why OOB Error Is Not Enough	11

4.2	10-Fold CV with caret	11
5	Confusion Matrix and Classification Evaluation	12
5.1	Confusion Matrix Structure	12
5.2	Key Metrics	12
5.2.1	Overall Accuracy	12
5.2.2	Cohen's Kappa	12
5.2.3	Sensitivity and Specificity (Binary)	12
5.2.4	F1 Score	13
6	Partial Dependence Plots	13
6.1	Definition	13
6.2	Using <code>pdp::partial()</code> with <code>ranger</code>	13
6.3	2-Way PDPs and Interaction Effects	14
7	Model Evaluation Framework	14
7.1	Regression Models: Selecting the Best	14
7.2	Classification Models: Multi-Class vs Binary	14
7.3	Diagnostic Plots	15
7.3.1	Observed vs Predicted (1:1 plot)	15
7.3.2	Residual Diagnostics	15
8	Spatial Prediction Workflow	15
8.1	Standard Pipeline	15
8.2	Naming Convention	15
8.3	Probability vs Class Maps	15
9	R Package Reference	16
10	Glossary	16
11	References	17

How to use this document: This companion guide provides the mathematical foundations, parameter descriptions, and implementation details for every method used in the *Tree Models Exercise*. Read a section before or after completing the corresponding exercise part. Cross-references to the exercise are noted throughout.

1 Classification and Regression Trees (CART)

1.1 Conceptual Overview

CART (Breiman et al., 1984) is a non-parametric supervised learning method that **recursively partitions** the predictor space into rectangular regions. Each partition is defined by a binary rule on a single predictor (e.g., `solar < 4200`). The result is a tree structure where:

- **Internal nodes** contain split rules
- **Branches** represent the outcome of a rule (left = TRUE, right = FALSE)
- **Leaf nodes (terminal nodes)** contain predictions

Trees are grown **greedily** — at each node the algorithm considers every possible split on every predictor and selects the one that maximally reduces impurity or error. There is no backtracking.

Key advantages for soil survey:

Property	Practical benefit
Non-parametric	No distributional assumptions; handles skewed soil data
Mixed input types	Accepts numeric, categorical, and ordered-factor predictors
Handles missing values	Uses surrogate splits; no imputation needed
Interpretable	Single tree diagram readable by field pedologists
Covariate selection built-in	Only informative splits appear in the tree

1.2 Split Criteria

1.2.1 Regression Trees — Residual Sum of Squares

For a **continuous** response (e.g., O horizon thickness `totot`), CART minimises the **Residual Sum of Squares (RSS)** at each candidate split (j, t) — feature j cut at threshold t :

$$\Delta RSS = RSS_{\text{parent}} - [RSS_{\text{left}} + RSS_{\text{right}}]$$

where $RSS = \sum_{i \in \text{node}} (y_i - \bar{y}_{\text{node}})^2$.

The prediction at each leaf is the **mean** of the training samples in that leaf:

$$\hat{y}_{\text{leaf}} = \bar{y}_{\text{leaf}} = \frac{1}{N_{\text{leaf}}} \sum_{i \in \text{leaf}} y_i$$

In R: `rpart(y ~ x, method = "anova")`

1.2.2 Classification Trees — Gini Impurity

For a **categorical** response (e.g., spodic intensity class, land cover), CART minimises **node impurity**. The default is **Gini impurity**:

$$G = 1 - \sum_{k=1}^K p_k^2$$

where p_k is the proportion of class k samples at the node. A perfectly pure node (all one class) has $G = 0$; maximum impurity (uniform distribution) has $G = 1 - 1/K$.

Entropy (information gain, used by C4.5/ID3) is an alternative:

$$H = - \sum_{k=1}^K p_k \log_2(p_k)$$

The split that minimises the **weighted average** impurity of the two child nodes is selected:

$$\text{Impurity}(j, t) = \frac{N_L}{N} G_L + \frac{N_R}{N} G_R$$

Gini vs entropy in practice: For most soil survey classification tasks, both criteria produce nearly identical trees. Gini is computationally faster (no logarithm). Entropy penalises near-pure nodes slightly more, which can sometimes produce slightly deeper trees on multi-class problems.

In R: `rpart(y ~ x, method = "class")` (default: Gini)

1.2.3 Multi-class vs Binary Classification

When the response has many classes (e.g., `spodint` with levels 0, 0.5, 1, 1.5, 2), CART must find splits that separate **five** distributions simultaneously. This is harder than binary classification because:

1. Rare intermediate classes (0.5, 1.5) provide weak signal at any single threshold
2. Adjacent classes in an ordered factor are environmentally similar, blurring decision boundaries
3. The class imbalance problem compounds — the majority class dominates splits

Collapsing to binary (spodic / non-spodic) removes the ambiguous intermediate boundaries. The pedological justification is that the spodic/non-spodic distinction maps to a clear process threshold (illuviation of organic matter and Al/Fe oxides) while the intermediate grades within spodic are more continuous.

Rule of thumb: If cross-validation error is substantially lower for a binary model, the intermediate classes are not well-supported by the available covariates and may be better handled as a regression response or discarded for mapping purposes.

1.3 Cost-Complexity Pruning

1.3.1 The Overfitting Problem

A fully-grown tree (no `cp` constraint) will eventually place every training observation in its own leaf — achieving zero training error but completely failing on new data. This is **overfitting**: the model has memorised noise rather than learned signal.

1.3.2 The Cost-Complexity Criterion

`rpart` controls tree complexity through the **complexity parameter** α (`cp` in R). The cost-complexity criterion is:

$$R_{\alpha}(T) = R(T) + \alpha \times |T|$$

where:

- $R(T)$ = training error (RSS for regression, misclassification rate for classification)
- $|T|$ = number of leaf nodes
- α = regularisation parameter (larger = simpler tree)

As α increases, the optimal tree gets smaller. `rpart` uses **10-fold cross-validation** internally to estimate $R_{\alpha}(T)$ for a range of α values and records results in the `cptable`.

1.3.3 Reading the CP Table

`printcp(model)` outputs:

Column	Meaning
<code>CP</code>	Complexity parameter value for this row
<code>nsplit</code>	Number of splits in the tree at this CP
<code>rel error</code>	Training error relative to root node error
<code>xerror</code>	Cross-validation error (relative)
<code>xstd</code>	Standard error of <code>xerror</code>

The **actual misclassification rate** for a classification tree is:

$$\text{error} = \text{Root Node Error} \times \text{xerror} \times 100\%$$

1.3.4 The 1-SE Rule

Rather than always selecting the tree with the minimum `xerror`, the **1-SE rule** (Breiman et al., 1984) selects the **simplest tree** whose CV error is within one standard error of the minimum. This prefers interpretability:

threshold = min(xerror) + xstd at minimum
optimal CP = largest CP where xerror ≤ threshold

```
cp_tbl <- model$cptable
xerr   <- cp_tbl[, "xerror"]
xstd   <- cp_tbl[, "xstd"]
min_idx <- which.min(xerr)
thresh <- xerr[min_idx] + xstd[min_idx]
cp_1se <- cp_tbl[max(which(xerr <= thresh)), "CP"]
pruned <- prune(model, cp = cp_1se)
```

Important: Use `max(which(...))` not `min(which(...))` — you want the row with the **largest CP** (smallest tree) that is still within the threshold. The CP table is ordered from largest to smallest CP, so rows near the top are simpler trees.

1.3.5 Visualising Pruning: `plotcp()`

`plotcp()` shows `xerror` (± 1 SE) vs tree size. The horizontal dashed line marks the 1-SE threshold. Any tree size below the line is a reasonable choice. Prefer the leftmost (simplest) crossing point.

1.4 Response Transformation: `log1p` for Skewed Data

1.4.1 When to Transform

O horizon thickness (`Otot`) and many other soil properties are **right-skewed**: the bulk of observations cluster near zero but a few extreme values (thick Histosol O horizons, peaty inclusions) stretch the distribution far to the right. In a regression tree:

1. The first split often isolates the outlier vs everything else, wasting the only top-level split
2. Node means are pulled toward outliers, distorting leaf predictions
3. RSS is dominated by the squared error on extreme observations

1.4.2 The `log1p` Transform

$$y^* = \ln(1 + y)$$

`log1p()` rather than `log()` is used because it is defined at $y = 0$ ($\log(1 + 0) = 0$), whereas $\log(0) = -\infty$.

The inverse (back-transform) is:

$$\hat{y} = e^{\hat{y}^*} - 1 = \text{expm1}(\hat{y}^*)$$

Use `expm1()` rather than `exp() - 1` to avoid catastrophic cancellation for values near zero.

1.4.3 Comparing Models on the Original Scale

Back-transformed predictions must be evaluated on the original scale for RMSE to be interpretable in cm:

```
pred_log_bt <- expm1(predict(model_log, newdata = val))
rmse_log    <- sqrt(mean((val$Otot - pred_log_bt)^2))
```

Caution: The log1p transform does **not** always improve RMSE. If the outlier is also present in the validation set, the un-transformed model may perform worse even after back-transformation, because the extreme value is genuinely hard to predict. The real benefit of the transform is more stable splits and better generalisation across the bulk of the distribution.

2 Random Forest

2.1 Bagging: The Foundation

Bootstrap AGGregating (Breiman, 1996) is the principle underlying Random Forest:

1. Draw B bootstrap samples from the training data (sampling with replacement, each sample $\approx 63\%$ of original data)
2. Fit a full decision tree T_b to each sample
3. Average predictions across all trees:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

For classification, use majority vote instead of averaging.

Why does averaging work? For B i.i.d. trees each with variance σ^2 :

$$\text{Var}(\bar{T}) = \frac{\sigma^2}{B} \xrightarrow{B \rightarrow \infty} 0$$

But tree predictions are **correlated** (all trained on similar data), so the actual variance reduction is:

$$\text{Var}(\hat{f}_{\text{bag}}) = \rho \sigma^2 + \frac{(1 - \rho) \sigma^2}{B}$$

As $B \rightarrow \infty$, the second term vanishes but the first term $\rho \sigma^2$ remains — this is the irreducible bagging variance.

2.2 Random Feature Subsampling: The RF Innovation

Breiman (2001) added one key modification: at **each split**, only a random subset of **mtry** features is considered (not all features). This **decorrelates** the trees, reducing ρ toward zero and making the $\rho \sigma^2$ floor much lower.

The result: Random Forest has strictly lower variance than bagging (all features at every split).

2.3 Key Parameters

2.3.1 ntree — Number of Trees

More trees = more stable predictions. The OOB error should **plateau** as a function of **ntree**. If the `plot(rf)` curve is still declining at your chosen **ntree**, increase it.

- Typical range: 500–2000 for soil survey datasets
- For grid search: start with 500, increase after finding optimal **mtry**

- Computational cost scales linearly with `ntree`

2.3.2 `mtry` — Features Tried per Split

Controls the degree of decorrelation:

Setting	Effect
<code>mtry = 1</code>	Extreme randomisation; high bias, low variance
<code>mtry = p</code>	Equivalent to bagging; all features eligible
<code>mtry = sqrt(p)</code>	Breiman's default for classification
<code>mtry = p/3</code>	Breiman's default for regression

For soil DSM with ~25 covariates ($p = 25$): default `mtry` 5 (classification) or 8 (regression). Optimal `mtry` is data-dependent — use grid search.

In R (`randomForest`): `mtry = 5`

In R (`ranger`): `mtry = 5`

2.3.3 `nodesize` / `min.node.size` — Minimum Leaf Size

Controls tree depth. Larger values = shallower trees = more bias, less variance.

Package	Argument	Default classification	Default regression
<code>randomForest</code>	<code>nodesize</code>	1	5
<code>ranger</code>	<code>min.node.size</code>	1	5

For small soil survey datasets ($n < 300$), increasing `nodesize` to 5–10 often helps prevent overfitting.

2.4 Out-of-Bag (OOB) Error

Each bootstrap sample leaves out ~37% of observations (the **out-of-bag** sample). Each tree predicts its own OOB observations. Aggregating these predictions across all trees gives a **free cross-validation estimate** of generalisation error — with no need for a separate hold-out set.

Accessing OOB error:

```
# randomForest - regression
rf$mse[rf$ntree]      # final OOB MSE
rf$rsq[rf$ntree]      # final OOB R²
sqrt(rf$mse[rf$ntree]) # OOB RMSE

# randomForest - classification
rf$err.rate[rf$ntree, "OOB"] # final OOB error rate

# ranger - any
sqrt(rf$prediction.error) # OOB RMSE (regression) or error rate (classification)
```

Spatial caution: OOB error assumes observations are independently drawn. For spatially-autocorrelated soil data, OOB error is **optimistic** — nearby training points can “leak” information across the bootstrap boundary. Always report spatially-blocked cross-validation for DSM deliverables.

2.5 Variable Importance

2.5.1 %IncMSE (Permutation Importance)

For each tree b and each predictor j :

1. Obtain OOB predictions normally $\rightarrow$ baseline OOB MSE_b
2. **Randomly permute** predictor j in the OOB data $\rightarrow$ permuted OOB $\text{MSE}_{b,j}$
3. Record the increase: $\text{MSE}_{b,j} - \text{MSE}_b$

Average across all trees:

$$\%IncMSE_j = \frac{1}{B} \sum_{b=1}^B \frac{\text{MSE}_{b,j} - \text{MSE}_b}{\text{MSE}_b} \times 100$$

Properties:

- Unbiased with respect to predictor cardinality (number of categories)
- Can be negative (a predictor whose permutation actually reduces error — possible in small samples)
- Preferred for scientific inference and reporting

2.5.2 IncNodePurity (Gini Importance)

Total RSS (regression) or Gini (classification) reduction attributable to splits on predictor j , summed across all nodes and all trees:

$$\text{IncNodePurity}_j = \sum_{b=1}^B \sum_{t:v(t)=j} \Delta\text{impurity}(t)$$

Properties:

- Fast to compute (recorded during tree-building, no re-evaluation needed)
- **Biased toward continuous predictors and high-cardinality categorical predictors** (more possible splits = more chances to accumulate importance)
- Useful for ranking but not for inferring the actual magnitude of effect

For soil survey publications: use `%IncMSE` / `MeanDecreaseAccuracy` and note in methods that permutation importance was used. Avoid `IncNodePurity` / `MeanDecreaseGini` in conclusions about predictor strength.

2.6 The Histosol Outlier Effect

A single extreme observation can inflate the OOB MSE and RSS, causing the model to allocate many splits to separate it from the bulk of the data. This is detectable by:

1. `hist(soildata$Otot)` — long right tail indicates a potential outlier
2. Comparing model performance with and without the observation
3. Large drop in MSE after removal confirms the observation was dominating the loss

The pedological decision: Histosols have a fundamentally different genesis (peat accumulation) than the other soil orders in the dataset. If the mapping goal is to predict O horizon thickness for mineral soils, removing Histosols is justified and should be documented. If the goal includes Histosols, a separate model or a Histosol presence/absence stratification may be more appropriate.

2.7 Hyperparameter Tuning

2.7.1 Grid Search

Exhaustive search over all combinations of hyperparameters:

```
hyper_grid <- expand.grid(
  mtry      = c(3, 5, 8, 10),
  nodesize  = c(1, 3, 5, 10),
  ntree     = c(200, 500, 1000)
)

for (i in seq_len(nrow(hyper_grid))) {
  model      <- randomForest(..., mtry = hyper_grid$mtry[i], ...)
  oob_err[i] <- model$err.rate[nrow(model$err.rate), "OOB"]
}
opt <- which.min(oob_err)
```

Computational cost: total models = $|mtry| \times |nodesize| \times |ntree|$. For $4 \times 4 \times 3 = 48$ combinations with $ntree = 500$, this is 24,000 trees. Use **ranger** for grid search — 5–20× faster than **randomForest**.

2.7.2 caret + ranger Tuning

`caret::train()` with `method = "ranger"` adds 10-fold CV to each hyperparameter combination, giving a more reliable tuning estimate than OOB alone:

```
tuning_grid <- expand.grid(
  mtry      = c(3, 5, 8),
  splitrule = "variance",
  min.node.size = c(3, 5, 10)
)

ctrl <- trainControl(method = "cv", number = 10, savePredictions = "final")

rf_tuned <- train(
  x = X, y = y,
  method      = "ranger",
  trControl   = ctrl,
  tuneGrid    = tuning_grid,
  num.trees   = 500
)

rf_tuned$bestTune  # optimal parameters
```

2.8 randomForest vs ranger

Feature	randomForest	ranger
Speed	Baseline	5–20× faster (C++ parallelism)
OOB error access	<code>\$err.rate[ntree, "OOB"]</code>	<code>\$prediction.error</code>
Importance argument	<code>importance = TRUE</code>	<code>importance = "permutation"</code>
Parallel trees	%dopar% required	Built-in (<code>num.threads</code>)
Large datasets	Memory-limited	Handles large n, p

Feature	randomForest	ranger
caret integration	method = "rf"	method = "ranger"

Practical guidance:

- Use `randomForest` for: exploratory work, `varImpPlot()`, `rfImpute()`
- Use `ranger` for: grid search, large DSM datasets, caret CV workflows
- Results should be numerically similar; minor differences from random seed

3 Feature Selection with Boruta

3.1 The Shadow Feature Approach

Boruta (Kursa & Rudnicki, 2010) is a wrapper around Random Forest that uses a hypothesis-testing framework to classify each predictor as *Confirmed*, *Tentative*, or *Rejected*.

Algorithm:

1. For each predictor j , create a **shadow copy** j^* by randomly permuting its values (destroying any relationship with the response)
2. Create an augmented dataset with both original and shadow predictors ($2p$ features)
3. Fit a Random Forest on the augmented dataset and record **MeanDecreaseAccuracy** for all features
4. Compare each original predictor's importance against the **maximum** shadow importance (the **maxImp** benchmark)
5. Features significantly above **maxImp** → **Confirmed**; significantly below → **Rejected**; inconclusive → **Tentative**
6. Repeat (**maxRuns** iterations), accumulating the binomial test outcomes

Key parameters:

Parameter	Default	Description
maxRuns	100	Maximum RF iterations
pValue	0.01	Significance threshold for accept/reject
doTrace	0	Verbosity (0 = silent, 1 = basic, 2 = detailed)

```
fs_bor <- Boruta(y = y, x = X, maxRuns = 50, doTrace = 1)

# Extract confirmed features
vars_confirmed <- getSelectedAttributes(fs_bor)

# Resolve tentative features (optional)
vars_final <- getSelectedAttributes(TentativeRoughFix(fs_bor))

# Importance statistics per feature
attStats(fs_bor)[order(-attStats(fs_bor)$meanImp), ]
```

Interpreting the plot:

- **Green boxes:** Confirmed important (above max-shadow benchmark)
- **Yellow boxes:** Tentative (insufficient evidence to accept or reject)
- **Red boxes:** Rejected (consistently below max-shadow)
- **Blue boxes:** Shadow features (benchmarks)

Boruta vs manual feature selection: Boruta is conservative — it accepts a feature only when there is strong statistical evidence of importance, not just correlation. For soil survey covariates derived from the same DEM (slope, aspect, TWI, etc.), highly correlated features often share the importance signal and Boruta may retain one while rejecting others. This is correct behaviour.

4 Cross-Validation

4.1 Why OOB Error Is Not Enough

OOB error assumes observations are drawn i.i.d. from the same population. Soil survey pedons are **spatially structured** — nearby sites share geology, climate, and land use. When the bootstrap sample contains sites 100 m from an OOB site, the model sees nearly the same environmental signal in training and validation. This produces **overly optimistic OOB error**.

k-fold cross-validation ($k = 5$ or 10) partitions the full dataset into k groups, trains on $k - 1$ and validates on 1, rotating until all observations have been held out exactly once. Spatial CV further ensures that folds do not contain nearby sites — critical for DSM and SSURGO update applications.

4.2 10-Fold CV with caret

The `caret::train()` function provides a standardised interface:

```
tc <- trainControl(
  method      = "cv",
  number      = 10,
  returnResamp = "all",      # save all fold predictions
  savePredictions = TRUE    # enables residual diagnostics
)

model_cv <- train(
  x      = X,      # predictor data frame
  y      = y,      # response vector
  method = "ranger", # or "rf", "rpart", etc.
  importance = "permutation",
  trControl = tc
)

# Results
model_cv$results      # RMSE, R², MAE per parameter combo
model_cv$bestTune     # optimal parameters
model_cv$finalModel   # final model fit on ALL data with best params
```

Metrics reported by caret:

Metric	Formula	Use
RMSE	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$	Primary accuracy metric (original units)
R ² (Rsquared)	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	Proportion variance explained
MAE	$\frac{1}{n} \sum y_i - \hat{y}_i $	Robust to outliers; interpretable in original units

Log-scale models: When the response was log1p-transformed, CV metrics from `caret` are on the **log scale**. To compare with un-transformed models, you must back-transform fold predictions manually or use a custom summary function.

5 Confusion Matrix and Classification Evaluation

5.1 Confusion Matrix Structure

A confusion matrix tabulates actual vs predicted classes. For binary classification:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

`caret::confusionMatrix()` computes the matrix and a full suite of statistics:

```
cm <- confusionMatrix(pred, reference, mode = "everything")
cm$overall      # Accuracy, Kappa, etc.
cm$byClass      # Sensitivity, Specificity, PPV, NPV, F1 per class
cm$table        # raw counts
```

5.2 Key Metrics

5.2.1 Overall Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Proportion of all predictions that are correct. Misleading when classes are imbalanced (a model predicting all non-spodic has high accuracy if 80% are non-spodic).

5.2.2 Cohen's Kappa

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

where P_o = observed agreement, P_e = expected agreement by chance. $\kappa = 1$ = perfect, $\kappa = 0$ = no better than chance, $\kappa < 0$ = worse than chance.

κ range	Interpretation
< 0.20	Slight
0.21–0.40	Fair
0.41–0.60	Moderate
0.61–0.80	Substantial
> 0.80	Almost perfect

5.2.3 Sensitivity and Specificity (Binary)

$$\text{Sensitivity (Recall)} = \frac{TP}{TP + FN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}$$

In soil survey context:

- **Sensitivity** = probability of correctly identifying a truly spodic site (producer's accuracy for the positive class)
- **Specificity** = probability of correctly identifying a truly non-spodic site

Which matters more? If the consequence of missing a spodic site (false negative) is worse than over-mapping (false positive), optimise for sensitivity. This is common in conservation-driven soil survey where under-mapping rare features has regulatory consequences.

5.2.4 F1 Score

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Harmonic mean of precision and recall. Useful for imbalanced classes where overall accuracy is uninformative.

6 Partial Dependence Plots

6.1 Definition

A **Partial Dependence Plot (PDP)** shows the marginal effect of one or two predictors on model predictions, averaging out the effect of all other predictors:

$$\hat{f}_j(x_j) = \frac{1}{N} \sum_{i=1}^N \hat{f}(x_j, \mathbf{x}_{-j,i})$$

For each value of x_j in the plot, the model is evaluated for every observation with that fixed value of x_j and its actual other predictor values. The mean prediction is plotted.

Note: PDPs assume **independence** between x_j and $\mathbf{x}_{-j}$. When predictors are correlated (e.g., elevation and temperature), the PDP may extrapolate into unrealistic regions of predictor space. Accumulated Local Effects (ALE) plots are a more robust alternative for correlated predictors.

6.2 Using `pdp::partial()` with `ranger`

`purrr::partial()` masks `pdp::partial()` when both packages are loaded. Always call explicitly:

```
pdp1 <- pdp::partial(
  rf_final,
  pred.var      = "landsatb7",    # predictor to vary
  plot          = FALSE,          # return data frame, not plot
  grid.resolution = 30,           # number of unique values to evaluate
  train         = soildata        # needed for ranger models
)

ggplot(pdp1, aes(x = landsatb7, y = yhat)) +
  geom_line(colour = "#2c5f2e", linewidth = 1.2) +
  geom_rug(data = soildata, aes(x = landsatb7, y = NULL),
    sides = "b", alpha = 0.3) +
  labs(x = "Landsat Band 7 Reflectance",
    y = "Partial Dependence (log1p 0 Horizon cm)") +
  theme_minimal()
```

Back-transforming PDPs for log-scale models:

```
pdp1$yhat_cm <- expm1(pdp1$yhat) # back to cm scale for interpretability
```

6.3 2-Way PDPs and Interaction Effects

```
pdp_2way <- pdp::partial(
  rf_final,
  pred.var      = c("landsatb7", "solar"),
  grid.resolution = 20,
  plot          = TRUE,
  plot.engine   = "ggplot2",
  train         = soildata
)
```

A 2-way PDP reveals **interactions** — when the effect of one predictor changes depending on the value of another. A flat surface indicates additivity; a curved or twisted surface indicates interaction.

7 Model Evaluation Framework

7.1 Regression Models: Selecting the Best

For O horizon thickness models, compare:

Model	Error metric	Scale
rpart full (raw)	Validation RMSE	cm
rpart pruned (raw)	Validation RMSE	cm
rpart pruned (log1p, back-transformed)	Validation RMSE	cm
RF raw (Histosol removed)	OOB RMSE	cm
RF log1p (back-transformed)	OOB RMSE	cm

Comparing OOB vs validation RMSE: OOB and hold-out RMSE are computed on different subsets and by different mechanisms. They are comparable in magnitude but should not be directly subtracted to infer improvement. All models should ideally be evaluated on the same held-out test set.

7.2 Classification Models: Multi-Class vs Binary

Metric	Multi-class spodint	Binary spodic/non-spodic
Overall accuracy	Lower	Higher
Kappa	Lower	Higher
Misclassification rate	Root Node Error × xerror	Root Node Error × xerror
Practical mapping use	Intensity gradient map	Presence/absence map

The binary model's higher accuracy does not mean it is always preferable. If the objective is to distinguish between low-spodic and high-spodic sites for management, the multi-class model provides information that the binary model discards.

7.3 Diagnostic Plots

7.3.1 Observed vs Predicted (1:1 plot)

```
plot(observed, predicted, pch = 16, col = "#2c5f2e",  
      xlab = "Observed Otot (cm)", ylab = "Predicted Otot (cm)")  
abline(0, 1, col = "red", lwd = 2) # perfect prediction line
```

Systematic bias appears as a cloud above (underprediction) or below (overprediction) the 1:1 line.

7.3.2 Residual Diagnostics

```
residuals <- observed - predicted  
  
plot(predicted, residuals, pch = 16) # heteroscedasticity check  
abline(h = 0, col = "red")  
  
qqnorm(residuals) # normality check  
qqline(residuals, col = "red")
```

A funnel-shaped residual plot (variance increasing with predicted value) is a common signature of a skewed response that benefits from log transformation.

8 Spatial Prediction Workflow

8.1 Standard Pipeline

```
Environmental covariates (rasters)  
↓  
Coregister to common CRS, resolution, extent (terra/raster)  
↓  
Stack into SpatRaster (terra::rast())  
↓  
Rename layers to match model predictor names exactly  
↓  
terra::predict(raster_stack, fitted_model, type = "response")  
↓  
Back-transform if response was log1p-transformed  
↓  
terra::writeRaster() → .tif for GIS
```

8.2 Naming Convention

```
names(raster_stack) == names(coef(model))[-1] # must match exactly
```

`terra::predict()` matches raster layer names to model predictor names. Any mismatch causes an error. If your raster files have different names than the column names in training data, rename either the layers or the model covariates.

8.3 Probability vs Class Maps

Output type	type =	Units	Use case
Class map	"response"	Factor (class label)	Final map product
Probability map	"prob"	0–1 per class	Uncertainty, threshold-based mapping
Vote count	"vote"	Integer per class	Ensemble inspection

Thresholding probability maps: Rather than using majority vote (implicit in `type = "response"`), you can apply a custom threshold to the probability surface to trade off sensitivity vs specificity:

```
LC_prob_spodic <- LC_prob[["spodic"]]      # probability of spodic class
LC_thresh      <- LC_prob_spodic > 0.4     # classify as spodic if P > 0.4
```

Lowering the threshold increases sensitivity (fewer missed spodic sites) at the cost of specificity (more non-spodic sites classified as spodic).

9 R Package Reference

Package	Function	Purpose
rpart	rpart(), prune(), printcp(), plotcp()	CART models and pruning
rpart.plot	rpart.plot(), prp()	Decision tree visualisation
randomForest	randomForest(), varImpPlot(), importance()	Classic RF
ranger	ranger(), importance()	Fast RF
caret	train(), trainControl(), confusionMatrix(), RMSE()	CV framework and metrics
Boruta	Boruta(), getSelectedAttributes(), attStats()	Feature selection
pdp	pdp::partial()	Partial dependence plots
vip	vip::vip()	Variable importance plots (ggplot2)
terra	rast(), predict(), writeRaster()	Raster prediction and output
sf	st_as_sf(), write_sf()	Spatial data handling
ggplot2	ggplot()	Publication-quality plots

10 Glossary

Term	Definition
OOB error	Out-of-bag error; free internal validation estimate in RF
mtry	Number of predictor candidates evaluated at each split
nodesize / min.node.size	Minimum observations required to split a node
CP (complexity parameter)	Regularisation term in rpart pruning
1-SE rule	Select simplest tree within 1 SE of minimum CV error

Term	Definition
Gini impurity	Node impurity measure for classification: $1 - \sum p_k^2$
%IncMSE	Permutation importance: increase in OOB MSE when predictor is shuffled
IncNodePurity	Gini/RSS reduction importance (biased; use %IncMSE for inference)
Bagging	Bootstrap aggregating: average many trees to reduce variance
Decorrelation	RF's <code>mtry</code> trick to reduce inter-tree correlation and further reduce variance
log1p transform	$\ln(1 + y)$; stabilises variance in right-skewed responses
expm1	$e^x - 1$; exact inverse of log1p; use instead of <code>exp(x) - 1</code>
Boruta	RF-based feature selection using shadow-feature hypothesis testing
PDP	Partial Dependence Plot; marginal effect of one predictor
Spatial CV	Cross-validation with geographically blocked folds
DSM	Digital Soil Mapping; spatial prediction of soil properties using covariates
CLORPT	Jenny's five soil-forming factors: Climate, Organisms, Relief, Parent material, Time

11 References

- Breiman, L., Friedman, J., Olshen, R., Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- James, G., Witten, D., Hastie, T., Tibshirani, R. (2021). *An Introduction to Statistical Learning (2nd ed.)*. Springer.
- Kurs, M. B., & Rudnicki, W. R. (2010). Feature selection with the Boruta package. *Journal of Statistical Software*, 36(11), 1–13.
- Nauman, T. W., Thompson, J. A., Odgers, N. P., & Libohova, Z. (2015). Predicting spodic properties for sandy soils using terrain and land cover information. *Soil Science Society of America Journal*, 79(1), 258–269.
- Wright, M. N., & Ziegler, A. (2017). ranger: A fast implementation of random forests for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1), 1–17.